

***Rust* Programlama Dili: Tasarım Felsefesi, Mimari Yapı ve Temel Paradigmalar**

Yazar: Resit Asrav

Özet

Bu çalışma, Mozilla Research tarafından 2010 yılında Graydon Hoare önderliğinde geliştirilen Rust programlama dilinin tasarım felsefesini, mimari yapısını ve temel paradigmalarını incelemektedir. Nesneye yönelik programlama kavramları, *fearless concurrency* modeli, fonksiyonel yapılar, ownership/borrowing temelli bellek yönetimi, cebirsel veri tipleri (ADT), sözdizim-anlambilim özellikleri ve generic programlama; Sebesta (2016), MacLennan (1999) ve Scott (2015) perspektifinden kapsamlı biçimde analiz edilmektedir.

Anahtar Kavramlar: Ownership, Borrow Checker, Fearless Concurrency, Zero-Cost Abstractions, ADT / Generics, Affine Tip Sistemi, RAIL.

1. Giriş ve Tarihçe

Programlama dillerinin tarihsel gelişimi, makine verimliliği ile yazılımcı üretkenliği arasındaki dengeyi bulma çabasıdır. C ve C++ gibi diller donanım üzerinde tam kontrol sağlarken bellek güvenliğini programcıya bırakır; **dangling pointer** (sarkan gösterici) ve **memory leak** (bellek sızıntısı) gibi kritik hatalar kaçınılmaz olur. Rust, 2010 yılında Graydon Hoare önderliğinde Mozilla bünyesinde doğmuş; çöp toplayıcı (GC) kullanmadan, derleme zamanı denetimleriyle bu hataları tamamen ortadan kaldırmayı hedefleyen statik tipli bir sistem programlama dilidir. 2022'den itibaren Linux çekirdeğine kabul edilen Rust, sistem programlamanın geleceği olarak konumlanmaktadır (Sebesta, 2016).

2. Nesneye Yönelik Kavramlar

Rust, klasik sınıf tabanlı NYP dillerinden (Java, C++) farklı bir yaklaşım benimser. Kalıtım (inheritance) kasıtlı olarak dışarıda bırakılmış; **Composition-over-inheritance** ilkesi uygulanır.

Veri yapıları struct, metotlar impl blokları ile tanımlanarak veri-davranış ayrımı korunur.

Polimorfizm ve arayüz mantığı trait'ler aracılığıyla sağlanır.

Rust



```
struct Daire { r: f64 }  
trait Alan { fn alan(&self) -> f64; }  
  
impl Alan for Daire {  
    fn alan(&self) -> f64 {  
        std::f64::consts::PI * self.r * self.r  
    }  
}
```

Kod 1 – Trait ile statik polimorfizm

dyn Trait ile dinamik dispatch, **impl Trait** ile statik dispatch desteklenir. Bu tasarım MacLennan (1999)'ın soyutlama ilkesini tip güvenliğiyle birleştirerek Go'nun interface yapısından daha ifade gücüne ulaşır.

3. Paralellik (Concurrency)

"**Fearless Concurrency**" (Korkusuz Eşzamanlılık) modeli ownership sistemi üzerine inşa edilmiştir. **Send** ve **Sync** marker trait'leri derleme zamanında veri yarışlarını (data race) statik olarak engeller (Scott, 2015, s. 614). Async/await sözdizimi ve Tokio runtime ile asenkron programlama da desteklenmektedir.

Rust



```
use std::{sync::{Arc, Mutex}, thread};  
  
let c = Arc::new(Mutex::new(0));  
let mut hs = vec![];  
  
for _ in 0..5 {  
    let c = Arc::clone(&c);  
    hs.push(thread::spawn(move || {  
        *c.lock().unwrap() += 1;  
    }));  
}
```

```
    }));  
}  
for h in hs { h.join().unwrap(); }
```

Kod 2 – Arc<Mutex> ile güvenli paralel sayaç

4. Fonksiyonel Yapılar

Rust birinci sınıf fonksiyon ve closure desteğiyle güçlü fonksiyonel programlama olanakları sunar. Değişkenler **varsayılan olarak immutable**'dir; mutabilite açıkça `mut` ile belirtilir. Bu tasarım Sebesta (2016)'nın yan etki minimizasyonu ilkesiyle örtüşür.

Rust



```
let nums = vec![1,2,3,4,5];  
let res: i32 = nums.iter()  
    .filter(|&x| x % 2 == 0)  
    .map(|&x| x * x)  
    .fold(0, |acc, x| acc + x);  
// res = 4 + 16 = 20
```

Kod 3 – Tembel iterator zinciri (lazy evaluation)

Iterator zinciri **tembel değerlendirme** (lazy evaluation) semantiğiyle çalışır; `.collect()` çağrılmadan ara koleksiyon oluşturulmaz. Yüksek-dereceli fonksiyonlar Haskell'deki list comprehension işlevini karşılar.

5. Binding ve Bellek Yönetimi

Rust'ın en yenilikçi olduğu alan bellek yönetimidir. **Ownership** modeli ve **RAII** deseni ile GC kullanmadan tam bellek güvenliği sağlanır. Her değerın tek sahibi vardır; sahip kapsam dışına çıkınca `Drop` devreye girerek bellek otomatik serbest bırakılır (MacLennan, 1999, s. 287).

- **Move Semantics:** Atamada sahiplik kopyalanmaz, devredilir.
- **Borrowing:** Aynı anda ya sınırsız `&T` (okuma) ya da tek `&mut T` (yazma) referansı verilebilir.

- **Lifetime** parametreleri referans geçerlilik sürelerini statik doğrular; dangling pointer olanaksız kılınır.

Rust



```
fn uzunluk(s: &String) -> usize {  
    s.len() // sahiplik geçmez  
}  
  
let s = String::from("Merhaba");  
println!("{}", uzunluk(&s));  
println!("{}", s); // hâlâ geçerli
```

Kod 4 – Borrowing (ödünç alma)

6. Ayırıştırıcı Özellik: Borrow Checker

Rust'u diğer sistem dillerinden en keskin ayıran özellik derleyici içindeki **Borrow Checker** mekanizmasıdır. **Affine tip sistemi** üzerine kurulu bu algoritma, use-after-free, double-free, null pointer dereference ve veri yarıışı hatalarını henüz derleme aşamasında reddeder; çalışma zamanına taşınmaz. C/C++'ın ham performansını Java'nın güvenliğiyle birleştiren tek ana akım sistem dilidir (Scott, 2015, s. 602).

7. Sözdizim ve Anlambilim

Sözdizimsel olarak C ailesine yakın olan Rust, anlambilimsel açıdan ML ve Haskell'e yakındır. Çoğu ifade birer değer döndürür (expression-oriented). match ile kapsamlı pattern matching; derleyici *exhaustiveness checking* ile tüm durumların ele alındığını zorunlu kılar.

Rust



```
fn bol(a:i32, b:i32) -> Result<i32,String> {  
    if b == 0 {  
        Err("Sıfıra bölme".into())  
    } else {  
        Ok(a / b)  
    }  
}
```

```
match bol(10, 2) {
    Ok(v) => println!("{}", v),
    Err(e) => println!("Hata: {e}"),
}
```

Kod 5 – Pattern matching ve Result<T,E>

? operatörü hata yayılımını zincirleme ifade eder. **Clone** açık derin kopya, **Copy** stack türleri için örtük bit-kopyayı belirtir.

8. ADT, Tipler ve Generics

Cebirsel Veri Tipleri (ADT), Rust'ta birinci sınıf dil özelliği olan **enum** ile uygulanır. "Milyar Dolarlık Hata" olarak bilinen Null kavramı yoktur; **Option<T>** varlık/yokluk durumunu, **Result<T,E>** hata yönetimini tip güvenli şekilde çözer (MacLennan, 1999, s. 312).

Rust

□□

```
enum Sekil {
    Daire(f64),
    Dikdortgen(f64, f64),
    Ucgen { taban: f64, yuksek: f64 },
}

fn yazdir<T: std::fmt::Display>(v: T) {
    println!("{}", v);
}
```

Kod 6 – ADT enum ve generic trait bound

Generic yapılar **monomorphization** ile derleme zamanında somutlaştırılır; çalışma zamanı maliyeti sıfırdır. Trait bounds (where T: Display + Clone) Haskell typeclass kısıtlamalarına işlevsel olarak eşdeğerdir (Scott, 2015).

9. Sonuç

Rust; sağladığı bellek güvenliği, sıfır maliyetli soyutlamalar, korkusuz eşzamanlılık ve modern tip sistemi ile sistem programlamanın geleceği olarak görülmektedir. Öğrenme eğrisi dik olsa da sunduğu statik garantiler büyük ölçekli ve kritik yazılımların geliştirilmesinde emsalsiz avantajlar sağlar. WebAssembly, gömülü sistemler ve işletim sistemi geliştirmede artan benimsenmesiyle Rust, önümüzdeki on yılın temel sistem dili olmaya adaydır.

Kaynakça

- [1] Sebesta, R. W. (2016). *Concepts of Programming Languages* (11. bs.). Pearson Education.
- [2] MacLennan, B. J. (1999). *Principles of Programming Languages: Design, Evaluation, and Implementation*. Oxford University Press.
- [3] Scott, M. L. (2015). *Programming Language Pragmatics* (4. bs.). Morgan Kaufmann / Elsevier.